

FRC Java Coding Lab 7.0 - Backbone Cuối Cùng Đã Đóng Bảng

Hợp đồng kiến trúc lâu dài cho Inheritance Development

1. Hợp đồng Backbone

Driver -> Xbox Controller -> controls -> commands -> subsystems -> io -> Hardware
telemetry quan sát mọi tầng nhưng không điều khiển pipeline

TRẠNG THÁI ĐÓNG BẢNG CUỐI CÙNG. Các bài học sau được phép mở rộng kiến trúc này nhưng không được thiết kế lại trách nhiệm, hướng dependency, luồng điều khiển hoặc luồng quan sát nếu chưa có Formal Architecture Review.

2. Trách nhiệm Package

Package	Câu hỏi chính	Trách nhiệm	Ví dụ	Không được chứa
controls	Driver muốn gì?	Chuyển input thô thành ý định điều khiển đã xử lý.	Deadband, đảo input, scaling, curve, slew-rate limiting.	Motor controller, CAN ID, CommandScheduler, logic phần cứng.
commands	Robot cần làm gì lúc này?	Điều phối một hành động hoặc chuỗi hành động của robot.	DefaultDriveCommand, ShootCommand, AutoBalanceCommand.	Cấu hình vendor, tạo motor, xử lý phần cứng cấp thấp.
subsystems	Mechanism làm được gì?	Cung cấp API cấp cao và sở hữu trạng thái mechanism hiện tại.	tankDrive(), stop(), runIntake(), setShooterSpeed().	Button mapping, truy cập Xbox, cấu hình vendor trực tiếp.
io	Phần cứng hoạt động như thế nào?	Định nghĩa hardware contract; implementation chứa vendor API và điền observation snapshot.	DriveIO, DriveIOInputs, DriveIOSparkMax, DriveIOSim, GyroIO.	Điều phối command, xử lý ý định driver, NetworkTables publisher.
telemetry	Robot đang làm gì?	Publish quan sát và chẩn đoán mà không thay đổi hành vi robot.	RobotTelemetry, DriveTelemetryFacade, typed NT4 publishers.	Điều khiển motor, xử lý input, scheduling, vendor hardware API.
util	Có thật sự dùng chung?	Chỉ chứa helper tổng quát dùng cho nhiều mechanism.	Math, units, diagnostics và fault helpers tổng quát.	Logic riêng của drive, intake, shooter hoặc mechanism.

3. Cây dự án ổn định

```
src/main/java/frc/robot
|- Main.java
|- Robot.java
|- RobotContainer.java
|- Constants.java
|- commands/
|  |- drive/
|  |- intake/
|  |- shooter/
|  `~ auto/
|- controls/
|- subsystems/
|- io/
|  |- drive/
|  |  |- DriveIO.java
|  |  |- DriveIOSparkMax.java
|  |  `~ DriveIOSim.java
|  |- gyro/
|  `~ vision/
|- telemetry/
|  |- RobotTelemetry.java
|  `~ drive/
```

```
| | - DriveTelemetryFacade.java
| | - DriveInputTelemetry.java
| | - DriveHardwareTelemetry.java
| | - DriveSensorTelemetry.java
| | - DrivePoseTelemetry.java
| | - DrivePoseTelemetry.java
| | - util/

src/main/deploy/pathplanner/
| - paths/
| - autos/
```

4. Quy tắc Dependency Bắt Buộc

- RobotContainer là composition root ngắn gọn: tạo object, chọn implementation, inject dependency, cấu hình command và binding.
- RobotContainer không chứa logic xử lý input, publish telemetry định kỳ hoặc hành vi phần cứng.
- Command phụ thuộc subsystem và telemetry facade nhỏ nhất mà nó thực sự cần.
- Subsystem chỉ phụ thuộc IO interface, không phụ thuộc vendor implementation.
- IO implementation được dùng vendor API; IO interface và Inputs snapshot không được phụ thuộc NetworkTables.
- Mỗi IO interface sở hữu một Inputs snapshot riêng theo mechanism. DriveIOInputs là implementation của pattern này cho drive.
- Hardware observation bắt nguồn từ IO. Mechanism state bắt nguồn từ subsystem hoặc estimator chuyên dụng.
- Telemetry chỉ publish snapshot; không đọc vendor device trực tiếp và không điều khiển hành vi robot.
- DriveTelemetryFacade che giấu các module telemetry nội bộ; caller không dùng getter chain sâu.
- util chỉ chứa logic tổng quát. Helper chỉ dùng cho một mechanism phải ở cùng mechanism đó.

5. Luồng được phê duyệt

CONTROL

Driver -> Controller -> controls -> command -> subsystem -> IO -> hardware

OBSERVATION

Controller / input processor -> DriveInputTelemetry

IO.updateInputs(MechanismIOInputs) -> subsystem periodic -> mechanism telemetry facade

Subsystem / estimator state -> pose or state telemetry

FORBIDDEN

Telemetry -X-> control / scheduling / vendor API

IO -X-> NetworkTables / command coordination

RobotContainer -X-> periodic business logic

6. Hợp đồng Drive Telemetry

- DriveInputTelemetry: raw, processed và commanded values của driver pipeline.
- DriveHardwareTelemetry: applied output, voltage, current, temperature, connection và fault từ DriveIOInputs.
- DriveSensorTelemetry: encoder và gyro observations qua IO/state contract đã được phê duyệt.
- DrivePoseTelemetry: pose và odometry state; class này không tự tính odometry.
- DriveTelemetryFacade: điểm truy cập công khai duy nhất của drive telemetry.

7. Chính sách Constants

- Constants.java vẫn là nơi quản lý cấu hình mặc định của dự án.
- Tổ chức constants theo mechanism hoặc nhóm chức năng bằng nested static classes như DriveConstants, OperatorConstants, AutoConstants, SensorConstants và TelemetryConstants.
- Không tách Constants.java trong các bài học thông thường.
- Chỉ được tạo package constants/ sau Formal Architecture Review và khi độ phức tạp dự án thực sự yêu cầu.

8. Chính sách Autonomous và PathPlanner

- commands/auto điều phối autonomous actions, command composition, factory và named-command registration.
- Tài nguyên PathPlanner nằm trong src/main/deploy/pathplanner/paths và src/main/deploy/pathplanner/autos.
- Autonomous command không sở hữu pose, odometry, state estimation hoặc coordinate state.
- Pose và estimation thuộc drive subsystem hoặc estimator chuyên dụng.

9. Chính sách Kế thừa Backbone

- Backbone đã đóng băng nhưng vẫn cho phép mở rộng.
- Mechanism mới phải tuân thủ ranh giới trách nhiệm, hướng dependency, control flow và observation flow đã được phê duyệt.
- Chỉ tạo class có trách nhiệm thực sự.
- Kế thừa responsibility pattern của drive; không sao chép máy móc mọi class của drive.

10. Per-Mechanism IO Inputs Pattern

- Mỗi IO interface sở hữu một Inputs snapshot riêng theo mechanism.
- Ví dụ: DriveIOInputs, IntakeIOInputs, ShooterIOInputs, ElevatorIOInputs, VisionIOInputs và GyroIOInputs.
- IO implementation điền observation; subsystem sử dụng snapshot; telemetry publish snapshot.
- Telemetry không đọc motor controller trực tiếp và subsystem không bỏ qua IO để đọc CAN device.

11. Chính sách Thay đổi Cuối cùng

Được phép: thêm class vào đúng package, mở rộng Inputs snapshot của mechanism, thêm facade method hoặc thêm mechanism mới theo pattern đã phê duyệt.

Không được phép trong bài học thông thường: di chuyển trách nhiệm giữa package, đảo hướng dependency, để telemetry điều khiển hành vi, để IO publish NetworkTables hoặc biến RobotContainer thành logic class.

ĐÃ KHÓA CUỐI CÙNG - KHÔNG CHỈNH SỬA BACKBONE THÊM